clipping pipeline - complete upload script: upload_youtube.py (page 1)

```
"""Upload approved Shorts to YouTube as PRIVATE videos.

    python upload_youtube.py --dry-run     show what would be uploaded, upload nothing
    python upload_youtube.py               upload

Only uploads videos YOU moved into output/approved/ (the .mp4 together with its .json).
Every video is uploaded as PRIVATE. This script cannot make anything public or unlisted.
After a successful upload both files move to output/uploaded/ and the YouTube ID is
written into the .json.

First-time setup: see README.md, section "YouTube upload".
"""
import argparse
import shutil
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
VENV = ROOT / ".venv"
if Path(sys.prefix).resolve() != VENV.resolve():
    sys.exit(subprocess.call([str(VENV / "Scripts" / "python.exe"), str(Path(__file__).resolve()), *sys.argv[1:]]))

from pipeline.core import registry                                  # noqa: E402
from pipeline.core.settings import APPROVED_DIR, CONFIG, OUTPUT     # noqa: E402
from pipeline.core.tools import read_json, write_json               # noqa: E402

PRIVACY = "private"                         # fixed on purpose; not read from any file or setting
SCOPES = ["https://www.googleapis.com/auth/youtube.upload"]
UPLOADED_DIR = OUTPUT / "uploaded"
CFG = CONFIG.get("upload", {})


def youtube_client():
    from google.auth.transport.requests import Request
    from google.oauth2.credentials import Credentials
    from google_auth_oauthlib.flow import InstalledAppFlow
    from googleapiclient.discovery import build

    secret = ROOT / CFG.get("client_secret", "secrets/client_secret.json")
    token = ROOT / CFG.get("token", "secrets/youtube_token.json")
    if not secret.exists():
        sys.exit(f"Missing {secret}. See README.md, section 'YouTube upload'.")
    creds = Credentials.from_authorized_user_file(str(token), SCOPES) if token.exists() else None
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            print("A browser window will open: sign in with the channel's Google account and allow uploads.")
            creds = InstalledAppFlow.from_client_secrets_file(str(secret), SCOPES).run_local_server(port=0)
        token.parent.mkdir(parents=True, exist_ok=True)
        token.write_text(creds.to_json(), encoding="utf-8")
    return build("youtube", "v3", credentials=creds)


def approved_items():
    items = []
    for mp4 in sorted(APPROVED_DIR.glob("*.mp4")):
        side = mp4.with_suffix(".json")
        if not side.exists():
            print(f"SKIP {mp4.name}: no matching .json sidecar (move both files together)")
            continue
        meta = read_json(side)
        if meta.get("youtube_video_id"):
            print(f"SKIP {mp4.name}: already uploaded as {meta['youtube_video_id']}")
            continue
        try:   # the source must still be approved for commercial use + derivatives
            registry.require_approved(meta["source"]["source_id"])
```

```
        except registry.SourceNotApproved as e:
            print(f"SKIP {mp4.name}: {e}")
            continue
        items.append((mp4, side, meta))
    return items


def body_for(meta):
    return {
        "snippet": {
            "title": meta["title"][:100],
            "description": meta["description"][:5000],
            "tags": CFG.get("tags", [])[:30],
            "categoryId": str(CFG.get("category_id", "27")),          # 27 = Education
            "defaultLanguage": CFG.get("language", "en"),
            "defaultAudioLanguage": CFG.get("language", "en"),
        },
        "status": {
            "privacyStatus": PRIVACY,
            "selfDeclaredMadeForKids": bool(CFG.get("made_for_kids", False)),
            "containsSyntheticMedia": bool(CFG.get("contains_synthetic_media", True)),
        },
    }


def upload(yt, mp4, meta):
    from googleapiclient.http import MediaFileUpload
    req = yt.videos().insert(part="snippet,status", body=body_for(meta),
                             media_body=MediaFileUpload(str(mp4), mimetype="video/mp4",
                                                        chunksize=8 * 1024 * 1024, resumable=True))
    resp = None
    while resp is None:
        status, resp = req.next_chunk()
        if status:
            print(f"\r  {int(status.progress() * 100)}%", end="", flush=True)
    print()
    return resp


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--dry-run", action="store_true")
    args = ap.parse_args()

    items = approved_items()[: int(CFG.get("max_per_run", 5))]
    if not items:
        print(f"Nothing to upload. Move a finished .mp4 and its .json from output/review to output/approved.")
        return 0
    for mp4, _, meta in items:
        print(f"{'WOULD UPLOAD' if args.dry_run else 'UPLOAD'} (private): {mp4.name} - {meta['title']}")
    if args.dry_run:
        return 0

    from googleapiclient.errors import HttpError
    yt = youtube_client()
    UPLOADED_DIR.mkdir(parents=True, exist_ok=True)
    failures = 0
    for mp4, side, meta in items:
        print(f"Uploading {mp4.name} ...")
        try:
            resp = upload(yt, mp4, meta)
        except HttpError as e:
            print(f"  FAILED: {e}")
            failures += 1
            if "youtubeSignupRequired" in str(e):
                print("  The Google account you signed in with has no YouTube channel (or you picked the")
                print("  personal account instead of the channel's Brand Account). See README, 'YouTube upload'.")
                return 1
            if "quota" in str(e).lower():
```

```
                print("  Daily YouTube API quota used up (about 6 uploads/day). Try again tomorrow.")
                return 1
            continue
        vid, privacy = resp["id"], resp.get("status", {}).get("privacyStatus")
        meta.update(youtube_video_id=vid, youtube_privacy=privacy,
                    uploaded_at=time.strftime("%Y-%m-%d %H:%M:%S"))
        write_json(side, meta)
        shutil.move(str(mp4), UPLOADED_DIR / mp4.name)
        shutil.move(str(side), UPLOADED_DIR / side.name)
        print(f"  done: https://studio.youtube.com/video/{vid}/edit  (privacy: {privacy})")
        if privacy != PRIVACY:
            print("  WARNING: YouTube reports a privacy other than private. Check it in YouTube Studio now.")
    return 1 if failures else 0


if __name__ == "__main__":
    sys.exit(main())
```